

Class Design Document: Vehicle Rental Management System

1. System Overview

The **Vehicle Rental Management System** is an object-oriented python application designed to manage vehicles, customer information and active or completed rental transactions. The system consists of json files to store data, input validation through try and except blocks and error handling.

2. Architecture & Class Relationships

Class Diagram Description

- **RentalSystem**: Coordinates overall system, handles the entire menu and file persistence. It maintains composition relationships containing lists of **Vehicle**, **Customer**, and **Rental** objects.
- **Vehicle**: Represents an individual vehicle.
- **Customer**: Stores personal details of a customer.
- **Rental**: Associates a **Vehicle** with a **Customer** using their respective IDs to track daily rates, duration, total costs and vehicle availability.

3. Class Specifications

3.1 Class: **Vehicle**

Represents a vehicle.

Attributes

- **vehicle_id** (**str**): Unique identifier for the vehicle.
- **registration** (**str**): license plate number.
- **make** (**str**): Manufacturer of the vehicle.
- **model** (**str**): Specific model name.

- `daily_rate` (`float`): Rental price per day.
- `is_available` (`bool`): Availability flag (`True` if the vehicle is available for rent, `False` if the vehicle is already rented out).

Methods

- `__init__(vehicle_id=None, registration=None, make=None, model=None, daily_rate=0.0, is_available=True)`: Initializes a new vehicle instance.
- `rent_a_vehicle()`: Sets `is_available` to `False`. When an individual vehicle object is called with this function, the car is rented out and thus no longer available.
- `return_a_vehicle()`: Sets `is_available` to `True`.
- `search_for_vehicle(search_term: str)`: Checks if `search_term` matches `vehicle_id` or `registration`. Prints details and returns `True` if matched, else `False`. Returns a boolean.
- `display_details()`: Prints details of the vehicle.
- `to_dictionary()`: Returns a dictionary representation of the vehicle object for JSON serialization.

3.2 Class: `Customer`

Represents a customer.

Attributes

- `customer_id` (`str`): Unique identifier for the customer.
- `customer_name` (`str`): Full name of the customer.
- `phone_number` (`str`): Primary contact phone number.
- `email` (`str`, optional): Customer email address (default: `None`).
- `driver_license` (`str`, optional): Driver's license identifier (default: `None`).

Methods

- `__init__(customer_id, customer_name, phone_number, email=None, driver_license=None)`: Validates that `customer_id`, `customer_name`, and `phone_number` are provided; raises `ValueError` if any are missing.
- `display_details()`: Prints customer profile information.
- `to_dictionary()`: Returns a dictionary representation of the customer object for JSON serialization.

3.3 Class: **Rental**

Represents a rental transaction linking a customer to a vehicle.

Attributes

- `rental_id (str)`: Unique identifier for the transaction.
- `vehicle_id (str)`: ID of the rented vehicle.
- `customer_id (str)`: ID of the renting customer.
- `days (int)`: Duration of the rental in days.
- `total_cost (float)`: Total charge calculated for the rental.
- `is_active (bool)`: Status flag (`True` if currently rented, `False` if returned).

Methods

- `__init__(rental_id, vehicle_id, customer_id, days, total_cost=0.0, is_active=True)`: Initializes rental object with these attributes.
- `calculate_cost(daily_rate: float)`: Calculates `total_cost = days * daily_rate`, updates `self.total_cost`, and returns it.
- `display_details()`: Displays summary of rental details and status.
- `to_dictionary()`: Returns a dictionary representation of the rental object for JSON serialization.

3.4 Class: **RentalSystem**

This class controls the functionality of the entire program. .

Attributes

- `vehicles (list[Vehicle])`: List of vehicles.
- `customers (list[Customer])`: List of customers.
- `rentals (list[Rental])`: list of rental records.

Methods

- `__init__()`: Initializes memory lists and invokes `load_data()`.
- `save_data()`: Serializes `vehicles`, `customers`, and `rentals` lists to JSON files (`vehicles.json`, `customers.json`, `rentals.json`).
- `load_data()`: Reads data from JSON files and reconstructs their object instances into memory lists. It also handles missing files through `FileNotFoundError` handling.
- `get_vehicle_by_id(v_id: str)`: Helper method to retrieve a vehicle by its ID.
- `get_customer_by_id(c_id: str)`: Helper method to retrieve a customer by their ID.
- `add_vehicle()`: Prompts user for vehicle attributes, validates input, ensures unique `vehicle_id`, and adds `Vehicle` object to collection.
- `display_vehicles()`: Displays details for all vehicles in the system.
- `search_vehicle()`: asks for search term and delegates matching logic to `Vehicle.search_for_vehicle()`.
- `register_customer()`: Prompts user to provide customer details, validates input, checks for duplicate IDs, and instantiates a `Customer` object.
- `display_customers()`: Displays details for all registered customers.
- `rent_vehicle()`: Orchestrates rental creation, verifying rental ID uniqueness, vehicle availability, customer existence and calculates total rental charges. Updates vehicle status via `rent_a_vehicle()`.

- `return_vehicle()`: Processes returning a vehicle by setting rental `is_active` to `False` and invoking `return_a_vehicle()` on the vehicle.
- `display_active_rentals()`: Filters and displays all active rental records.

4. Data Persistence Schema

The application uses JSON files to persist entities between program executions:

- **Vehicle Schema (`vehicles.json`)**: Stores `vehicle_id`, `registration`, `make`, `model`, `daily_rate`, and `is_available`.
- **Customer Schema (`customers.json`)**: Stores `customer_id`, `customer_name`, `phone_number`, `email`, and `driver_license`.
- **Rental Schema (`rentals.json`)**: Stores `rental_id`, `vehicle_id`, `customer_id`, `days`, `total_cost`, and `is_active`.